

CM5474 LOS SAMPLE QUESTION BANK

UNIT 1 (CO1)		
Sr. No.	Question	
1	Enlist the services of Linux operating system. (R)	
2	Recall the term process and its life cycle. (R)	
3	List the basic features of Linux operating system(R)	
4	Summarize any four features of shell (U)	
5	Recall the term Linux Distribution and Enlist popular Linux distributions (R)	
6	Describe Linux operating system and explain its silent features. (U)	
7	Explain the different type of Linux shell.(A)	
8	Explain the basic architecture of Linux operating System with neat labelled diagram (U/A)	
9	Explain the features of Linux Operating System.(U)	
10	Describe components of Linux Operating system (Kernel, System Library, System Utility) (U)	
11	Recall the Term System call in Linux	
12	Enlist types of system calls with example	
13	Describe Operating System services in details.	
14	Describe following services provided by operating system i) Program execution ii) I/O Operation iii) File system manipulation iv) Communication	
15	Describe following services provided by operating system i) Error handling ii) Resource Management iii) Protection iv) security	
16	Describe run levels in Linux	
17	Explain the process to change run level in Linux	
18	Describe Linux file Structure by specifying different directories and their functions	
19	Explain types of Linux file system	
20	Recall the term shell. Enlist different shell types.(R/U)	

1. Enlist the services of Linux operating system. (R)

The Linux operating system provides the following services:

- **Program Execution** – Loads and runs programs.
 - **I/O Operations** – Manages input/output devices.
 - **File System Manipulation** – Handles file storage, retrieval, and permissions.
 - **Communication** – Enables inter-process communication via shared memory or message passing.
 - **Error Detection** – Detects and handles system errors.
 - **Resource Allocation** – Manages CPU, memory, and device allocation.
 - **Protection & Security** – Ensures user authentication and access control.
-

2. Recall the term process and its life cycle. (R)

A **process** is a running instance of a program, including its execution context, memory, and resources.

Process Life Cycle States:

1. **New** – Process is being created.
2. **Ready** – Process is waiting to be assigned a CPU.
3. **Running** – Process is executing on CPU.
4. **Waiting** – Process is waiting for I/O or some event.
5. **Terminated** – Process has completed execution.
6. **Suspended** – Process is paused but still in memory.

A process moves between these states based on scheduling and resource availability.

3. List the basic features of Linux operating system. (R)

- **Portable** – Works on different hardware.
 - **Open-source** – Free to use and modify.
 - **Multi-user** – Multiple users can operate simultaneously.
 - **Multiprogramming** – Can run multiple programs concurrently.
 - **Hierarchical File System** – Standardized directory structure.
 - **Shell** – Command-line interpreter for user interaction.
 - **Security** – Password protection, encryption, and access control.
-

4. Summarize any four features of shell. (U)

- **Command Execution** – Accepts and processes user commands.
 - **Wildcard Substitution** – Allows pattern-based file name matching.
 - **I/O Redirection** – Redirects input/output to/from files or devices.
 - **Piping** – Allows output of one command to be used as input for another.
-

5. Recall the term Linux Distribution and Enlist popular Linux distributions. (R)

A **Linux Distribution (Distro)** is an OS package including the Linux kernel, system libraries, and applications.

Popular Linux Distributions:

- **Ubuntu**
 - **Debian**
 - **Fedora**
 - **Linux Mint**
 - **openSUSE**
 - **Arch Linux**
 - **Kali Linux**
-

6. Describe Linux operating system and explain its salient features. (U)

Linux is an open-source, Unix-like operating system known for stability, security, and flexibility.

Salient Features:

- **Open-source development**
 - **Highly secure environment**
 - **Stable with high performance**
 - **Supports multitasking and multi-user access**
-

7. Explain the different types of Linux shells. (A)

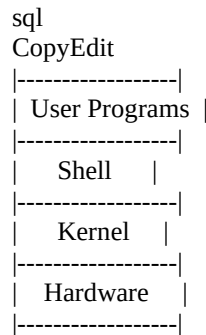
- **Bourne Shell (sh)** – Original shell, lacks interactive features.
 - **Bash (Bourne Again Shell)** – Default shell with advanced scripting.
 - **C Shell (csh)** – C-like syntax, good for scripting.
 - **Korn Shell (ksh)** – Combination of Bourne and C shell features.
 - **Tcsh (TENEX C Shell)** – Enhanced version of C shell.
-

8. Explain the basic architecture of Linux OS with a diagram. (U/A)

Linux OS Architecture:

1. **Hardware Layer** – Physical devices like CPU, RAM, HDD.
2. **Kernel** – Manages hardware resources, provides core functions.
3. **Shell** – Interface between user and kernel.
4. **Utilities** – System tools and applications.

Diagram:



9. Explain the features of Linux OS. (U)

Refer to question 3 for features like **multi-user support, security, portability, and stability**.

10. Describe components of Linux OS (Kernel, System Library, System Utility). (U)

- **Kernel** – Core component managing hardware and processes.
 - **System Library** – Provides standard functions for applications.
 - **System Utility** – Programs performing specific OS tasks.
-

11. Recall the Term System Call in Linux.

A **system call** is the interface between applications and the kernel, allowing programs to request OS services like file handling, process control, and communication.

12. Enlist types of system calls with examples.

1. **Process Control** – fork(), exec(), exit().
 2. **File Management** – open(), read(), write(), close().
 3. **Device Management** – ioctl(), read(), write().
 4. **Information Maintenance** – getpid(), getuid(), setuid().
 5. **Communication** – pipe(), shmget(), msgsend().
 6. **Protection** – chmod(), chown(), setuid().
-

13. Describe OS services in detail.

See question 1.

14. Describe OS services (Program execution, I/O, File system, Communication).

Covered in question 1.

15. Describe OS services (Error handling, Resource Management, Protection, Security).

Covered in question 1.

16. Describe run levels in Linux.

- **0** – Halt
 - **1** – Single-user mode
 - **2** – Multi-user mode (No NFS)
 - **3** – Multi-user mode (CLI)
 - **4** – Unused (customizable)
 - **5** – Multi-user mode (GUI)
 - **6** – Reboot
-

17. Explain the process to change run level in Linux.

Use the **telinit** command:

telinit <runlevel>

Example:

```
telinit 3 # Switch to CLI mode
```

Check current run level:

```
who -r  
runlevel
```

18. Describe Linux file structure by specifying different directories and their functions.

Refer to **FHS (Filesystem Hierarchy Standard)**.

- **/** – Root directory
 - **/bin/** – Essential binaries
 - **/etc/** – Configuration files
 - **/home/** – User directories
 - **/var/** – Logs, spool files
 - **/usr/** – User applications
 - **/tmp/** – Temporary files
-

19. Explain types of Linux file systems.

- **Ext4** – Default modern Linux file system.
 - **XFS** – High-performance file system for large storage.
 - **Btrfs** – Advanced file system with snapshot support.
 - **ReiserFS** – Alternative to Ext3 with better performance.
 - **JFS** – IBM's journaling file system.
-

20. Recall the term shell. Enlist different shell types. (R/U)

A **shell** is a command-line interface for interacting with the OS.
See question 7 for shell types like **Bash**, **C shell**, **Korn shell**, **Tcsh**.

Let me know if you need more explanations! 🚀

UNIT 2 (CO2)

1	State the function of 1)echo 2)passwd command (R)														
2	State the function of ordinary file handling command.(R)														
3	State the use of cal command & date command.														
4	Recall the term i) directory ii) home directory														
5	Enlist different file access types available in Linux and its symbols														
	<table><tr><td>Access type</td><td>Symbol</td><td>What it means</td></tr><tr><td>Read</td><td>r</td><td>Allows displaying, coping and compiling a file</td></tr><tr><td>Write</td><td>w</td><td>Allows editing and deleting of a file</td></tr><tr><td>Execute</td><td>x</td><td>Allows execution of a file</td></tr></table>			Access type	Symbol	What it means	Read	r	Allows displaying, coping and compiling a file	Write	w	Allows editing and deleting of a file	Execute	x	Allows execution of a file
Access type	Symbol	What it means													
Read	r	Allows displaying, coping and compiling a file													
Write	w	Allows editing and deleting of a file													
Execute	x	Allows execution of a file													
6	Recall the use of u,g and o options associated with linux files														
	<table><tr><td>Symbols</td><td>Description</td></tr><tr><td>u</td><td>The user or owner of file and directories</td></tr><tr><td>g</td><td>Members of the group o which user belongs</td></tr><tr><td>o</td><td>All other system users (users who are neither u or g)</td></tr></table>			Symbols	Description	u	The user or owner of file and directories	g	Members of the group o which user belongs	o	All other system users (users who are neither u or g)				
Symbols	Description														
u	The user or owner of file and directories														
g	Members of the group o which user belongs														
o	All other system users (users who are neither u or g)														
7	Give example of one internal and one External command (R)														
8	State the use of help command in linux														
9	Demonstrate how to get current file permission and modify it.														
10	Demonstrate how modify current file permission using Octal method														
11	Expalin the use of tr command in linux														
12	Differentiate between grep, egrep and fgrep commands														
13	Explain the use following linux commands by giving suitable example i)egrep ii)fgrep														
14	Explain file attribute of file permission in Linux Operating System. (A)														
15	Explain grep command with example. (A)														
16	Differentiate between internal and external command in Linux. (U)														
17	Classify Linux file types and give its uses. (Ordinary File or Regular File, Directory File , Device File , Symbolic link file or soft link file , FIFO (Named Pipe) , Socket)														
18	Explain general purpose utilities commands in Linux (cal, date, who, whoami, tty, uname, passwd, echo, tput, be,script, wall, write, mail) (U)														
19	Summarize File Naming Conventions use for Linux system														
20	Describe parent child relationship in Linux file system and Enlist common directories in Linux and give their uses.(U)														

21	Describe the term path and explain absolute path and relative path with suitable example (A)	
22	Demonstrate use of following commands in Linux (pwd, cd, mkdir, rmdir, ls, cat, cp)	
23	Demonstrate use of following commands in Linux (rm, mv, cmp, comm, diff)	

1. State the function of:

1) *echo command*

- The echo command **displays a line of text or a variable's value**.
- It is useful for printing messages, formatting output, and debugging shell scripts.

Example:

```
echo "Hello, Linux!"
```

Output:

```
Hello, Linux!
```

- **Options:**
 - -e : Enables interpretation of backslash escapes like \n (new line), \t (tab).
 - -n : Prevents a new line after output.

Example:

```
echo -e "Hello\nWorld"
```

Output:

```
Hello
World
```

Source:

2) *passwd command*

- The passwd command **changes the password of a user**.
- The system prompts for the **current password**, **new password**, and **confirmation**.

Example:

```
passwd
```

Output:

Changing password for user.
(Current) UNIX password:
New password:
Retype new password:

- **Only the root user** can change other users' passwords.
 - Users can only change their own passwords. **Source:**
-

2. State the function of ordinary file handling commands.

Ordinary file handling commands help manage and manipulate files.

Command	Function
cat	View or create files.
cp	Copy files and directories.
mv	Move or rename files.
rm	Remove files or directories.
cmp	Compare two files byte by byte.
diff	Show differences between two files.

Example of cat command:

```
cat myfile.txt
```

Source:

3. State the use of cal command & date command.

1) cal command (Calendar)

- Displays a calendar for the **current month**, a **specific month**, or a **specific year**.

Example:

```
cal 2024
```

Output:

```
python-repl
2024
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
  7  8  9 10 11 12 13
...
```

- **Options:**
 - -y : Show the full year.
 - -m : Start week from Monday.

2) *date command*

- Displays the **current date and time**.

Example:

```
date
```

Output:

```
Wed Mar 6 15:30:45 IST 2024
```

- **Options:**
 - +%Y-%m-%d : Show date in YYYY-MM-DD format.
 - +%T : Show time as HH:MM:SS.

Source:

4. Recall the term:

i) *Directory*

- A **directory** is a **folder** that stores files and subdirectories.
- Linux follows a **hierarchical file system**.
- Example: /home/user/Documents

ii) *Home Directory*

- The **default directory** for a user upon login.
- Identified using ~ (tilde).

Example:

```
cd ~
pwd
```

Output:

/home/username

Source:

5. Enlist different file access types available in Linux and their symbols.

Access Type	Symbol	Meaning
Read	r	View file contents.
Write	w	Modify or delete file.
Execute	x	Run a file as a program.

Example:

```
ls -l myfile.txt
```

Output:

```
-rwxr-xr-- 1 user user 1024 Mar 6 10:30 myfile.txt
```

Source:

6. Recall the use of u, g, and o options associated with Linux files.

Symbol	Description
u	Owner (User)
g	Group
o	Others

Example: Change permissions using chmod:

```
chmod g+w myfile.txt
```

This **adds write permission** for the group. **Source:**

7. Give an example of one internal and one external command.

Type	Example	Description
Internal	cd	Change directory.
External	ls	List files in a directory.

Example:

```
type cd
```

Output:

```
cd is a shell builtin
```

```
type ls
```

Output:

```
ls is /bin/ls
```

Source:

8. State the use of the help command in Linux.

- Displays information about **built-in shell commands**.
- Useful for checking **syntax and options**.

Example:

```
help cd
```

Source:

9. Demonstrate how to get current file permissions and modify them.

Check file permissions:

```
ls -l myfile.txt
```

Modify permissions (Symbolic Method):

```
chmod u+x myfile.txt
```

Source:

10. Demonstrate how to modify current file permissions using the Octal method.

- **Octal values for permissions:**

- 7 = rwx
- 6 = rw-
- 5 = r-x
- 4 = r--
- 0 = ---

Example:

```
chmod 755 myfile.txt
```

Source:

11. Explain the use of the `tr` command in Linux.

- `tr` is used for **character translation**.

Example: Convert lowercase to uppercase

```
echo "hello world" | tr '[a-z]' '[A-Z]'
```

Output:

```
HELLO WORLD
```

Source:

12. Differentiate between `grep`, `egrep`, and `fgrep`.

Command	Function
<code>grep</code>	Searches for a pattern using basic regex.

Command	Function
egrep	Uses extended regex (Supports `
fgrep	Fixed string search (No regex support).

Example:

```
grep "word" file.txt  
egrep "word1|word2" file.txt  
fgrep "exact" file.txt
```

Source:

13. Explain the use of `egrep` and `fgrep` with examples.

- `egrep` Example:

```
egrep "error|failed" logfile.txt
```

- `fgrep` Example:

```
fgrep "error" logfile.txt
```

Source:

UNIT 3 (CO3)

	Recall the term vi and it's modes of working	
	Explain the role of wild card characters in Linux. Describe different wild card characters in Linux by giving example.	
	State the use of echo command? Give suitable example	
	Recall the term shell variables and enlist rules applicable for shell variables	
	Recall the term Referencing variable and demonstrate its uses	
	Explain command line arguments in shell script with suitable example (U)	
	Explain 'expr' command with suitable example	
	Write shell script to print greatest of three numbers entered by the user.(A)	
	Design a shell script to find factorial of a number	
	Design a shell script to print Fibonacci series	
	Design a shell script to find whether entered number is prime or not	
	Design a shell script to find whether entered number is even or odd	
	Design a shell script to find whether entered number is divisible by 5 and 7	
	Develop a shell script that prompts user to enter name of the directory to be created .If directory already exist prompt the message "Directory already exist". If directory does not exist create the new directory and prompt the message "Directory created successfully"	
	Develop a shell script to prompt the user to enter 3 numbers. Find the greatest and smallest number.	
	Demonstrate the syntax of while...do loop in shell script by giving example	
	Demonstrate the syntax of The for.....do loop in shell script by giving example	
	Demonstrate the syntax of The if conditional loop in shell script by giving example	
	Explain The Case Statement in shell script with suitable example	
	Develop a shell script to check whether a file is an ordinary file, directory, readable/writable. Use command line argument to read filename	

1. Recall the term vi and its modes of working

The vi editor is a **text editor** in Linux used for creating and modifying files. It operates in three **modes**:

1. **Command Mode** (Default Mode)
 - Used for navigation, text deletion, copying, pasting, and saving.
 - Examples:
 - dd → Delete a line.
 - yy → Yank (copy) a line.
 - p → Paste copied content.
2. **Insert Mode** (For text input)
 - Switch using:
 - i → Insert before the cursor.
 - a → Insert after the cursor.
 - o → Open a new line.
3. **Ex (Colon) Mode** (For saving and quitting)
 - :w → Save.
 - :q → Quit.
 - :wq or :x → Save and quit.

Source:

2. Explain the role of wildcard characters in Linux and give examples.

Wildcards allow users to match **multiple filenames** based on patterns.

Common Wildcards:

Wildcard	Description	Example
*	Matches zero or more characters	ls *.txt (Lists all .txt files)
?	Matches exactly one character	ls file?.txt (Matches file1.txt, file2.txt)
[]	Matches any single character in brackets	ls file[12].txt (Matches file1.txt and file2.txt)
{ }	Matches multiple patterns	ls {file1,file2}.txt (Matches file1.txt, file2.txt)

Source:

3. State the use of echo command with an example.

- echo prints messages or variable values on the terminal.

- It is **used in shell scripting**.

Example 1: Print a message

```
echo "Hello, Linux!"
```

Output:

Hello, Linux!

Example 2: Print a variable

```
name="John"  
echo "Welcome $name!"
```

Output:

Welcome John!

Source:

4. Recall the term shell variables and enlist rules applicable for shell variables.

Shell Variables

- Variables in Linux store **values** like numbers, strings, or file paths.
- Assigned using = without spaces.

Example:

```
name="Linux"  
echo "I love $name"
```

Output:

I love Linux

Rules for Shell Variables

1. Must **start with a letter** or `_`, not a number.
2. Cannot have **spaces** (use `_` instead).
3. Case-sensitive (`NAME` $\neq$ `name`).
4. No special characters (`$`, `@`, `!`, etc.).
5. Use `export` to make global.

Source:

5. Recall the term Referencing Variable and demonstrate its uses.

Referencing a variable means **accessing its value** using \$.

Example:

```
greeting="Hello, Linux"  
echo $greeting
```

Output:

Hello, Linux

Source:

6. Explain command-line arguments in shell scripts with an example.

Command-line arguments allow users to pass **values to a script**.

Example:

```
#!/bin/bash  
echo "First Argument: $1"  
echo "Second Argument: $2"
```

Run script:

```
./script.sh Linux Bash
```

Output:

```
First Argument: Linux  
Second Argument: Bash
```

Source:

7. Explain expr command with an example.

expr is used for **arithmetic calculations**.

Example:

```
expr 10 + 5
```

Output:

```
15
```

Source:

8-14: Shell Script Examples

8. Greatest of three numbers

```
#!/bin/bash
echo "Enter three numbers: "
read a b c
if [ $a -gt $b ] && [ $a -gt $c ]; then
    echo "$a is greatest"
elif [ $b -gt $c ]; then
    echo "$b is greatest"
else
    echo "$c is greatest"
fi
```

9. Factorial of a number

```
#!/bin/bash
echo "Enter a number: "
read num
fact=1
for (( i=1; i<=num; i++ )); do
    fact=$((fact * i))
done
echo "Factorial: $fact"
```

10. Fibonacci Series

```
#!/bin/bash
echo "Enter number of terms: "
read n
a=0
b=1
echo "Fibonacci series: "
for (( i=0; i<n; i++ )); do
    echo -n "$a "
```

```
temp=$((a + b))
a=$b
b=$temp
done
```

11. Check Prime Number

```
#!/bin/bash
echo "Enter a number: "
read num
for (( i=2; i<num; i++ )); do
    if [ $((num % i)) -eq 0 ]; then
        echo "$num is not prime"
        exit
    fi
done
echo "$num is prime"
```

12. Check Even or Odd

```
#!/bin/bash
echo "Enter a number: "
read num
if [ $((num % 2)) -eq 0 ]; then
    echo "Even"
else
    echo "Odd"
fi
```

13. Check divisibility by 5 and 7

```
#!/bin/bash
echo "Enter a number: "
read num
if [ $((num % 5)) -eq 0 ] && [ $((num % 7)) -eq 0 ]; then
    echo "Divisible by 5 and 7"
else
    echo "Not divisible by 5 and 7"
fi
```

14. Check if directory exists, if not create it

```
#!/bin/bash
echo "Enter directory name: "
read dir
if [ -d "$dir" ]; then
    echo "Directory already exists"
```

```
else
  mkdir "$dir"
  echo "Directory created successfully"
fi
```

15-19: Shell Scripting Constructs

15. Syntax of while loop

```
#!/bin/bash
count=1
while [ $count -le 5 ]; do
  echo "Count: $count"
  ((count++))
done
```

16. Syntax of for loop

```
#!/bin/bash
for i in 1 2 3 4 5; do
  echo "Number: $i"
done
```

17. Syntax of if statement

```
#!/bin/bash
echo "Enter a number: "
read num
if [ $num -gt 10 ]; then
  echo "Greater than 10"
else
  echo "Less than or equal to 10"
fi
```

18. Case Statement Example

```
#!/bin/bash
echo "Enter choice (start/stop): "
read choice
case $choice in
  start) echo "Starting service";;
  stop) echo "Stopping service";;
  *) echo "Invalid choice";;
esac
```

19. Check if a file is ordinary, directory, readable, or writable

```
#!/bin/bash
echo "Enter filename: "
read file
if [ -d "$file" ]; then
    echo "This is a directory"
elif [ -f "$file" ]; then
    echo "This is an ordinary file"
    [ -r "$file" ] && echo "File is readable"
    [ -w "$file" ] && echo "File is writable"
fi
```